STEP SEMESTER 3 · CATEGORY C

Abstraction, Interface & Class vs Interface

Complete Assignment Answer Key

Included sections
Part A – 5 Java coding problems
Part B – 10 multiple-choice questions
Part C – 5 conceptual questions

Solutions follow the problem statements, business rules, and sample formats in the supplied CodInClub / BridgeLabz assignment.

Prepared as a study and submission reference.

Part A – Coding Problems

Implementation notes

Each solution is a standalone Java program. Save each program in a .java file matching its public class name. Compile and run one program at a time. The solutions use abstract base classes and overridden methods so the reporting code can work with parent-class references.

Problem 1 – Garden Plot Area Report

Design explanation

Use an abstract Plot class to store the owner's name and require each shape to implement calculateArea(). The report loop works with Plot references, so a new shape can be added by creating another subclass without changing the reporting logic.

Complete Java solution

```
import java.util.*;

abstract class Plot {
    protected String owner;
    public Plot(String owner) { this.owner = owner; }
    public abstract double calculateArea();
    public String getOwner() { return owner; }
    public abstract String getShape();
}

class CirclePlot extends Plot {
    private double radius;
    public CirclePlot(String owner, double radius) {
        super(owner); this.radius = radius;
    }
    public double calculateArea() { return Math.PI * radius * radius; }
    public String getShape() { return "CIRCLE"; }
}

class RectanglePlot extends Plot {
    private double length, width;
    public RectanglePlot(String owner, double length, double width) {
        super(owner); this.length = length; this.width = width;
    }
    public double calculateArea() { return length * width; }
    public String getShape() { return "RECTANGLE"; }
}

class TrianglePlot extends Plot {
    private double base, height;
    public TrianglePlot(String owner, double base, double height) {
        super(owner); this.base = base; this.height = height;
    }
    public double calculateArea() { return 0.5 * base * height; }
    public String getShape() { return "TRIANGLE"; }
}

public class GardenPlotAreaReport {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int n = sc.nextInt();
        List<Plot> plots = new ArrayList<>();
        for (int i = 0; i < n; i++) {
            String shape = sc.next().toUpperCase();

            String owner = sc.next();
            Plot p;
            switch (shape) {
                case "CIRCLE":
                    p = new CirclePlot(owner, sc.nextDouble()); break;
                case "RECTANGLE":
                    p = new RectanglePlot(owner, sc.nextDouble(),
                                          sc.nextDouble()); break;
                case "TRIANGLE":
                    p = new TrianglePlot(owner, sc.nextDouble(),
                                         sc.nextDouble()); break;
                default: throw new IllegalArgumentException("Unknown shape");
            }
            plots.add(p);
        }
        double total = 0.0;
        for (Plot p : plots) {
            double area = p.calculateArea();
            total += area;
            System.out.printf(Locale.US, "%s (%s): %.2f%n",
                              p.getOwner(), p.getShape(), area);
        }
        System.out.printf(Locale.US, "Total Area: %.2f%n", total);
        sc.close();
    }
}
```

Sample input

```
3
CIRCLE Asha 5
RECTANGLE Ravi 4 6
TRIANGLE Neha 10 3
```

Expected output

```
Asha (CIRCLE): 78.54
Ravi (RECTANGLE): 24.00
Neha (TRIANGLE): 15.00
Total Area: 117.54
```

Problem 2 – Weekly Staff Pay

Design explanation

Staff is abstract so a generic staff member cannot be created. Each concrete staff type implements calculatePay(). HourlyStaff applies the normal rate up to 40 hours and 1.5 times the rate to additional hours.

Complete Java solution

```
import java.util.*;

abstract class Staff {
    protected String name;
    public Staff(String name) { this.name = name; }
    public String getName() { return name; }
    public abstract double calculatePay();
}

class FullTimeStaff extends Staff {
    private double weeklySalary;
    public FullTimeStaff(String name, double weeklySalary) {
        super(name); this.weeklySalary = weeklySalary;
    }
    public double calculatePay() { return weeklySalary; }
}

class HourlyStaff extends Staff {
    private double hours, rate;
    public HourlyStaff(String name, double hours, double rate) {
        super(name); this.hours = hours; this.rate = rate;
    }
    public double calculatePay() {
        if (hours <= 40) return hours * rate;
        return 40 * rate + (hours - 40) * rate * 1.5;
    }
}

class Intern extends Staff {
    private double stipend;
    public Intern(String name, double stipend) {
        super(name); this.stipend = stipend;
    }
    public double calculatePay() { return stipend; }
}

public class WeeklyStaffPay {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int n = sc.nextInt();
        List<Staff> staffList = new ArrayList<>();
        for (int i = 0; i < n; i++) {
            String type = sc.next().toUpperCase();
            String name = sc.next();

            Staff s;
            switch (type) {
                case "FULLTIME":
                    s = new FullTimeStaff(name, sc.nextDouble()); break;
                case "HOURLY":
                    s = new HourlyStaff(name, sc.nextDouble(),
                                        sc.nextDouble()); break;
                case "INTERN":
                    s = new Intern(name, sc.nextDouble()); break;
                default: throw new IllegalArgumentException("Unknown staff type");
            }
            staffList.add(s);
        }
        double total = 0.0;
        for (Staff s : staffList) {
            double pay = s.calculatePay();
            total += pay;
            System.out.printf(Locale.US, "%s: %.2f%n", s.getName(), pay);
        }
        System.out.printf(Locale.US, "Total Payroll: %.2f%n", total);
        sc.close();
    }
}
```

Sample input

```
3
FULLTIME Asha 12000
HOURLY Ravi 45 200
INTERN Neha 5000
```

Expected output

```
Asha: 12000.00
Ravi: 9500.00
Neha: 5000.00
Total Payroll: 26500.00
```

Problem 3 – Library Late Fine Counter

Design explanation

LibraryItem provides the common title and days-late data while requiring each item type to calculate its fine. DVD fines are capped at 50 using Math.min().

Complete Java solution

```
import java.util.*;

abstract class LibraryItem {
    protected String title;
    protected int daysLate;
    public LibraryItem(String title, int daysLate) {
        this.title = title; this.daysLate = daysLate;
    }
    public String getTitle() { return title; }
    public abstract double calculateFine();
}

class Book extends LibraryItem {
    public Book(String title, int daysLate) { super(title, daysLate); }
    public double calculateFine() { return daysLate * 2.0; }
}

class DVD extends LibraryItem {
    public DVD(String title, int daysLate) { super(title, daysLate); }
    public double calculateFine() {
        return Math.min(daysLate * 5.0, 50.0);
    }
}

class Magazine extends LibraryItem {
    public Magazine(String title, int daysLate) { super(title, daysLate); }
    public double calculateFine() { return daysLate * 1.0; }
}

public class LibraryLateFineCounter {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int n = sc.nextInt();
        List<LibraryItem> items = new ArrayList<>();
        for (int i = 0; i < n; i++) {
            String type = sc.next().toUpperCase();
            String title = sc.next();
            int days = sc.nextInt();
            LibraryItem item;
            switch (type) {
                case "BOOK": item = new Book(title, days); break;
                case "DVD": item = new DVD(title, days); break;
                case "MAGAZINE": item = new Magazine(title, days); break;
                default: throw new IllegalArgumentException("Unknown item type");

            }
            items.add(item);
        }
        double total = 0.0;
        for (LibraryItem item : items) {
            double fine = item.calculateFine();
            total += fine;
            System.out.printf(Locale.US, "%s: %.2f%n",
                              item.getTitle(), fine);
        }
        System.out.printf(Locale.US, "Total Fines: %.2f%n", total);
        sc.close();
    }
}
```

Sample input

```
3
BOOK Algebra 4
DVD Inception 12
MAGAZINE Sports 3
```

Expected output

```
Algebra: 8.00
Inception: 50.00
Sports: 3.00
Total Fines: 61.00
```

Problem 4 – Electricity Connection Billing

Design explanation

ElectricityConnection stores the units used and defines an abstract bill calculation. Home, Shop, and Factory implement their own tariff rules. Factory uses Math.max() to enforce the minimum bill of 1000.

Complete Java solution

```
import java.util.*;

abstract class ElectricityConnection {
    protected double units;
    public ElectricityConnection(double units) { this.units = units; }
    public abstract double calculateBill();
}

class HomeConnection extends ElectricityConnection {
    public HomeConnection(double units) { super(units); }
    public double calculateBill() {
        if (units <= 100) return units * 5.0;
        return 100 * 5.0 + (units - 100) * 7.0;
    }
}

class ShopConnection extends ElectricityConnection {
    public ShopConnection(double units) { super(units); }
    public double calculateBill() { return units * 8.0 + 100.0; }
}

class FactoryConnection extends ElectricityConnection {
    public FactoryConnection(double units) { super(units); }
    public double calculateBill() {
        return Math.max(units * 6.0, 1000.0);
    }
}

public class ElectricityConnectionBilling {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int n = sc.nextInt();
        List<ElectricityConnection> connections = new ArrayList<>();
        List<String> types = new ArrayList<>();
        for (int i = 0; i < n; i++) {
            String type = sc.next().toUpperCase();
            double units = sc.nextDouble();
            ElectricityConnection c;
            switch (type) {
                case "HOME": c = new HomeConnection(units); break;
                case "SHOP": c = new ShopConnection(units); break;
                case "FACTORY": c = new FactoryConnection(units); break;
                default: throw new IllegalArgumentException("Unknown type");
            }

            connections.add(c);
            types.add(type);
        }
        double total = 0.0;
        for (int i = 0; i < n; i++) {
            double bill = connections.get(i).calculateBill();
            total += bill;
            System.out.printf(Locale.US, "%s: %.2f%n", types.get(i), bill);
        }
        System.out.printf(Locale.US, "Total: %.2f%n", total);
        sc.close();
    }
}
```

Sample input

```
3
HOME 150
SHOP 90
FACTORY 120
```

Expected output

```
HOME: 850.00
SHOP: 820.00
FACTORY: 1000.00
Total: 2670.00
```

Problem 5 – Travel Booking with a Common Fee

Design explanation

Abstract Booking owns the single shared booking fee and adds it in one final method. Each travel mode implements only its base-fare calculation. Changing the common fee requires editing only BOOKING_FEE.

Complete Java solution

```
import java.util.*;

abstract class Booking {
    private static final double BOOKING_FEE = 50.0;
    protected double distanceKm;
    public Booking(double distanceKm) { this.distanceKm = distanceKm; }
    protected abstract double calculateBaseFare();

    public final double calculateTotal() {
        return calculateBaseFare() + BOOKING_FEE;
    }
}

class BusBooking extends Booking {
    public BusBooking(double distanceKm) { super(distanceKm); }
    protected double calculateBaseFare() { return distanceKm * 2.0; }
}

class TrainBooking extends Booking {
    public TrainBooking(double distanceKm) { super(distanceKm); }
    protected double calculateBaseFare() { return distanceKm * 1.5; }
}

class FlightBooking extends Booking {
    public FlightBooking(double distanceKm) { super(distanceKm); }
    protected double calculateBaseFare() {
        return 2500.0 + distanceKm * 4.0;
    }
}

public class TravelBookingWithCommonFee {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int n = sc.nextInt();
        List<Booking> bookings = new ArrayList<>();
        List<String> modes = new ArrayList<>();
        for (int i = 0; i < n; i++) {
            String mode = sc.next().toUpperCase();
            double distance = sc.nextDouble();
            Booking b;
            switch (mode) {
                case "BUS": b = new BusBooking(distance); break;
                case "TRAIN": b = new TrainBooking(distance); break;
                case "FLIGHT": b = new FlightBooking(distance); break;

                default: throw new IllegalArgumentException("Unknown mode");
            }
            bookings.add(b);
            modes.add(mode);
        }
        for (int i = 0; i < n; i++) {
            System.out.printf(Locale.US, "%s: %.2f%n",
                    modes.get(i), bookings.get(i).calculateTotal());
        }
        sc.close();
    }
}
```

Sample input

```
3
BUS 200
TRAIN 300
FLIGHT 500
```

Expected output

```
BUS: 450.00
TRAIN: 500.00
FLIGHT: 4550.00
```

Part B – Quiz Answer Key

Choose the single best answer for each question.

Q1. Which statement best describes abstraction?

Correct answer: C
Abstraction hides implementation details and exposes what an object does.

Q2. Which statement about an abstract class in Java is true?

Correct answer: C
An abstract class cannot be instantiated directly. It may contain abstract methods, concrete methods, or both.

Q3. A concrete subclass fails to implement an inherited abstract method. What happens?

Correct answer: B
The subclass must itself be declared abstract; otherwise, the program does not compile.

Q4. Which modifier is not allowed for an abstract method?

Correct answer: C – private
An abstract method must be implemented by a subclass, so it cannot be private.

Q5. Interface fields are implicitly:

Correct answer: C – public, static, and final
Fields declared in an interface are constants, accessible as public static final.

Q6. How many interfaces can one Java class implement?

Correct answer: C – Any number
A class may implement multiple interfaces, even though it can extend only one class.

Q7. What access modifier must an implemented interface method use?

Correct answer: D – public
Interface abstract methods are public, so the implementing method cannot reduce visibility.

Q8. Why can Java classes implement multiple interfaces but extend one class?

Correct answer: C
The assignment's intended answer is that interfaces do not contribute inherited instance state, avoiding conflicting data inheritance associated with the diamond problem.

Q9. Two interfaces provide the same default method and the class does not override it. What happens?

Correct answer: D
The compiler reports a conflict; the class must explicitly override the method and resolve the ambiguity.

Q10. Which statement about constructors is true?

Correct answer: B
An abstract class can have a constructor. It runs as part of constructing a concrete subclass object.

Part C – Concept Questions

Question 1 – Explain abstraction with a real-life example.

A smartphone camera is a useful example. The user sees controls such as focus, zoom, and capture, and receives a photograph. The user does not need to know how the sensor converts light into electrical signals, how image processing reduces noise, or how the file is encoded. Abstraction presents the operations needed by the user while hiding the internal implementation.

Question 2 – Difference between abstraction and encapsulation; how do they work together?

Abstraction focuses on what an object does and hides unnecessary implementation details. Encapsulation focuses on bundling data with the methods that operate on it and restricting direct access to that data. For example, a BankAccount class can expose deposit(), withdraw(), and getBalance() as its simple public interface (abstraction), while keeping balance private and validating every change inside its methods (encapsulation). Together they reduce complexity and protect the object's state.

Question 3 – Why can an abstract class have a constructor, and when does it execute?

An abstract class can have a constructor to initialize common fields and enforce setup shared by all its subclasses. Although the abstract class cannot be instantiated directly, its constructor executes when a concrete subclass object is created. Java runs the superclass constructor as part of the subclass construction process, before the subclass constructor body completes.

Question 4 – Why can an abstract method not be private, static, or final?

Private methods are not visible to subclasses, so a subclass cannot override them to provide the required implementation. Static methods belong to the class rather than an individual object and are hidden, not overridden polymorphically. Final methods cannot be overridden by subclasses. Since an abstract method requires a concrete subclass implementation through overriding, these modifiers conflict with its purpose.

Question 5 – What is the diamond problem, and how does Java address it?

The diamond problem occurs in multiple inheritance when a class inherits the same method or state through two parent paths, making it unclear which implementation or inherited state should be used. Java avoids this ambiguity for classes by allowing a class to extend only one class. It permits multiple interfaces because interfaces do not add ordinary per-object instance fields. If two interfaces provide conflicting default methods with the same signature, the implementing class must override the method and resolve the conflict explicitly.

End of Answer Key

Check that the file names used for Java source files match the public class names before compiling.